

Univerzitet u Beogradu
Matematički fakultet



Stefan Jevtić

Rešavanje problema bojenja grafova pomoću DSatur
heuristike

seminarski rad

Beograd, 2026.

Sadržaj

1	Uvod	2
2	Teorijske osnove	2
2.1	Grafovi i osnovni pojmovi	3
2.2	Problem bojenja grafova	3
2.3	Računska složenost problema	3
2.4	Heuristički pristupi bojenju grafova	3
3	DSatur heuristika	4
3.1	Osnovna ideja algoritma	4
3.2	Koraci algoritma	4
3.3	Vremenska složenost	5
4	Implementacija	5
4.1	Format ulaznih podataka	5
4.2	Arhitektura rešenja	6
4.3	Standardna implementacija	6
4.4	Implementacija sa binarnim hipom	6
4.5	Implementacija sa Fibonačijevim hipom	7
4.6	Poređenje implementacija	8
5	Vizuelizacija rada algoritma	8
5.1	Grafički korisnički interfejs	8
5.2	Animacija procesa bojenja	9
5.3	Prikaz metrika algoritma	11
5.4	Zanimljivosti tokom razvoja aplikacije	11
6	Zaključak	12

1 Uvod

Grafovi predstavljaju jedan od najznačajnijih modela za predstavljanje odnosa između objekata i imaju široku primenu u različitim oblastima računarstva, matematike, telekomunikacija, logistike i optimizacije. Jedan od najpoznatijih problema u teoriji grafova jeste problem bojenja grafova, koji podrazumeva dodeljivanje boja čvorovima grafa tako da nijedna dva susedna čvora nemaju istu boju, pri čemu je cilj koristiti najmanji mogući broj boja. Ovaj problem pripada klasi NP-teških problema, zbog čega pronalaženje optimalnog rešenja za velike grafove predstavlja značajan izazov.

Zbog velike računske složenosti, u praksi se često primenjuju heuristički algoritmi koji omogućavaju pronalaženje kvalitetnih, iako ne nužno optimalnih, rešenja u prihvatljivom vremenu. Jedna od najpoznatijih heuristika za rešavanje problema bojenja grafova jeste DSatur odnosno heuristika zasnovana na meri zasićenja čvorova grafa (eng. *Degree Saturation*), koju je 1979. godine predložio Danijel Brelaz [1]. Osnovna ideja ovog algoritma zasniva se na izboru neobojenog čvora sa najvećim stepenom zasićenja, odnosno najvećim brojem različitih boja koje se već pojavljuju u njegovom neposrednom susedstvu. Na ovaj način algoritam uspešno usmerava proces bojenja ka efikasnom korišćenju raspoloživih boja.

Predmet ovog seminarskog rada jeste detaljno razmatranje DSatur heuristike, zajedno sa njenim implementacionim varijantama. Poseban akcenat stavljen je na način rada algoritma, kriterijume izbora čvorova, vremensku složenost različitih implementacija i mogućnosti njihove optimizacije primenom struktura podataka kao što su binarni i Fibonaccijev hip. Pored toga, rad obuhvata i opis razvoja interaktivne desktop aplikacije namenjene vizuelizaciji rada algoritma, koja omogućava korisniku da korak po korak prati proces bojenja grafa, promene stepena zasićenja čvorova i donošenje odluka tokom izvršavanja algoritma.

Cilj rada je da kroz teorijski pregled i praktičnu implementaciju prikaže značaj heurističkih metoda u rešavanju kombinatornih optimizacionih problema, kao i da demonstira kako vizuelizacija algoritama može doprineti njihovom lakšem razumevanju i analizi. Interaktivni prikaz rada DSatur heuristike omogućava detaljan uvid u svaki korak algoritma, čime predstavlja koristan edukativni alat za proučavanje problema bojenja grafova i savremenih tehnika optimizacije.

2 Teorijske osnove

U ovom poglavlju predstavljeni su osnovni pojmovi iz teorije grafova neophodni za razumevanje problema bojenja grafova i DSatur heuristike. Pored formalne definicije problema, razmatrana je njegova računska složenost, kao i razlozi zbog kojih se u praksi primenjuju heuristički algoritmi.

2.1 Grafovi i osnovni pojmovi

Graf predstavlja strukturu podataka koja se sastoji od skupa čvorova V i skupa grana E koje povezuju parove čvorova [5, 2]. Graf se označava kao

$$G = (V, E),$$

gde je V konačan skup čvorova, a $E \subseteq V \times V$ skup grana. Graf je usmeren ukoliko svaka grana ima definisan smer, odnosno predstavlja uređeni par čvorova. U suprotnom, graf je neusmeren. U okviru ovog rada razmatraju se isključivo neusmereni grafovi.

Dva čvora nazivaju se susednim ukoliko između njih postoji grana. Stepen čvora predstavlja broj grana incidentnih tom čvoru i označava se sa $\deg(v)$. Ovi pojmovi predstavljaju osnovu za definisanje problema bojenja grafova i heuristika koje se koriste za njegovo rešavanje.

2.2 Problem bojenja grafova

Problem bojenja grafova podrazumeva dodeljivanje boja čvorovima grafa tako da nijedna dva susedna čvora nemaju istu boju. Formalno, potrebno je odrediti funkciju

$$c : V \rightarrow \{1, 2, \dots, k\},$$

tako da za svaku granu $(u, v) \in E$ važi

$$c(u) \neq c(v).$$

Najmanji broj boja kojim je moguće pravilno obojiti dati graf naziva se hromatski broj grafa i označava se sa $\chi(G)$. Pronalazak tog broja predstavlja osnovni cilj problema bojenja grafova.

Problem bojenja ima značajnu primenu u brojnim oblastima, uključujući raspoređivanje zadataka, dodelu frekvencija u komunikacionim sistemima, izradu rasporeda ispita i planiranje korišćenja ograničenih resursa.

2.3 Računska složenost problema

Određivanje hromatskog broja proizvoljnog grafa predstavlja NP-težak problem. Ne postoji poznat algoritam koji ovaj problem rešava u polinomskom vremenu za sve grafove, zbog čega egzaktni algoritmi postaju nepraktični za grafove većih dimenzija.

Iz tog razloga u praksi se često koriste aproksimativni i heuristički algoritmi koji ne garantuju optimalno rešenje, ali omogućavaju pronalaženje kvalitetnih rešenja uz znatno manju računsku složenost.

2.4 Heuristički pristupi bojenju grafova

Heuristike za bojenje grafova zasnivaju se na izboru redosleda u kome će čvorovi biti obrađeni. Kvalitet dobijenog rešenja u velikoj meri zavisi upravo od ovog redosleda.

Najjednostavniji pohlepni algoritam obrađuje čvorove unapred zadatim redosledom i svakom čvoru dodeljuje najmanju raspoloživu boju koja nije iskorišćena kod njegovih već obojenih suseda. Iako je njegova vremenska složenost mala, broj korišćenih boja može biti znatno veći od optimalnog.

Naprednije heuristike određuju redosled obrade dinamički, na osnovu trenutnog stanja bojenja [4]. Jedna od najuspešnijih među njima jeste DSatur heuristika, koja prioritet daje čvorovima sa najvećim stepenom zasićenja, odnosno najvećim brojem različitih boja prisutnih u njihovom susedstvu. Ovakav pristup često daje rešenja veoma bliska optimalnim uz prihvatljivo vreme izvršavanja.

3 DSatur heuristika

DSatur predstavlja jednu od najpoznatijih heuristika za rešavanje problema bojenja grafova. Algoritam dinamički određuje redosled obrade čvorova na osnovu trenutnog stanja bojenja, čime se postiže efikasnije korišćenje raspoloživih boja u odnosu na jednostavne pohlepne pristupe. U ovom poglavlju opisana je osnovna ideja algoritma, način njegovog izvršavanja i analiza vremenske složenosti različitih implementacija.

3.1 Osnovna ideja algoritma

Naziv algoritma potiče od stepena zasićenja čvora, koji predstavlja broj različitih boja dodeljenih njegovim već obojenim susedima. Za razliku od klasičnih pohlepnih algoritama, kod kojih je redosled obrade unapred određen, DSatur u svakom koraku ponovo određuje naredni čvor za bojenje na osnovu trenutnog stanja grafa.

Intuicija algoritma zasniva se na tome da se prioritet daje čvorovima čiji je izbor boje najviše ograničen, odnosno onima koji imaju najveći stepen zasićenja. Na taj način smanjuje se verovatnoća nepotrebnog uvođenja novih boja u kasnijim fazama izvršavanja algoritma.

3.2 Koraci algoritma

Izvršavanje algoritma može se podeliti na nekoliko uzastopnih koraka:

1. Svim čvorovima dodeljuje se početni stepen zasićenja jednak nuli
2. Bira se neobojeni čvor sa najvećim stepenom zasićenja ili sa najvećim brojem neobojenih suseda. Ukoliko postoji više kandidata, prednost se daje čvoru sa leksikografski manjom vrednošću
3. Izabranom čvoru dodeljuje se najmanja boja koja nije dodeljena nijednom od njegovih obojenih suseda
4. Ažuriraju se stepeni zasićenja svih neobojenih susednih čvorova
5. Postupak se ponavlja sve dok svi čvorovi ne budu obojeni

3.3 Vremenska složenost

Vremenska složenost DSatur algoritma zavisi od načina izbora narednog čvora za bojenje.

Standardna implementacija u svakom koraku pretražuje skup svih neobojenih čvorova kako bi pronašla čvor sa najvećim stepenom zasićenja. Zbog toga ukupna vremenska složenost iznosi

$$\mathcal{O}(n^2),$$

gde je n broj čvorova grafa.

Primenom odgovarajućih struktura podataka moguće je ubrzati izbor narednog čvora. Implementacija zasnovana na binarnom hipu ostvaruje vremensku složenost

$$\mathcal{O}((n + m) \log n),$$

dok implementacija koja koristi Fibonačijev hip postiže složenost

$$\mathcal{O}(m + n \log n),$$

gde je m broj grana grafa.

Iako sve implementacije primenjuju identičnu heuristiku i daju isti način bojenja, razlikuju se u efikasnosti održavanja prioriteta čvorova tokom izvršavanja algoritma.

4 Implementacija

U ovom poglavlju opisana je implementacija aplikacije za vizuelizaciju rada DSatur heuristike. Posebna pažnja posvećena je organizaciji programskog rešenja, implementaciji različitih varijanti algoritma i strukturama podataka koje utiču na efikasnost njegovog izvršavanja.

4.1 Format ulaznih podataka

Grafovi koje aplikacija obrađuje zadati su u tekstualnim datotekama. Svaka datoteka sadrži opis jednog neusmerenog grafa u jednostavnom formatu, što omogućava lako kreiranje i učitavanje novih primera.

U prvom redu datoteke nalaze se dva cela broja: broj čvorova n i broj grana m . Nakon toga sledi m redova, pri čemu svaki red sadrži dva cela broja u i v koji predstavljaju granu između čvorova u i v .

Primer ulazne datoteke prikazan je u nastavku:

```
6 9
1 4
1 5
1 6
```

2 4
2 5
2 6
3 4
3 5
3 6

Prvi red označava da graf sadrži šest čvorova i devet grana. Svaki naredni red definiše jednu neusmerenu granu između odgovarajućeg para čvorova. U prikazanom primeru opisan je potpuni bipartitni graf $K_{3,3}$.

Čvorovi su numerisani prirodnim brojevima od 1 do n . Prilikom učitavanja ulaznih podataka ignorišu se petlje, odnosno grane koje povezuju čvor sa samim sobom.

4.2 Arhitektura rešenja

Aplikacija je razvijena u programskom jeziku Python i organizovana modularno, pri čemu su pojedine funkcionalnosti razdvojene u zasebne celine. Ovakva organizacija omogućava jednostavnije održavanje, nadogradnju sistema i ponovno korišćenje zajedničkih komponenti. Za predstavljanje i obradu grafova korišćena je struktura podataka koju obezbeđuje biblioteka *NetworkX* [3].

Sve implementacije DSatur algoritma nasleđuju zajedničku baznu klasu koja sadrži funkcionalnosti za inicijalizaciju grafa, održavanje metapodataka o čvorovima i beleženje koraka izvršavanja. Na ovaj način izbegnuto je dupliranje koda, dok se razlike između implementacija svode isključivo na način izbora narednog čvora za bojenje.

Pored algoritamskog dela, aplikacija obuhvata modul za učitavanje grafova iz tekstualnih datoteka i grafički korisnički interfejs zadužen za prikaz procesa bojenja.

4.3 Standardna implementacija

Standardna implementacija predstavlja direktnu realizaciju DSatur heuristike. U svakom koraku prolazi kroz skup svih neobojenih čvorova i određuje čvor sa najvećim stepenom zasićenja. Ukoliko postoji više kandidata, prednost ima čvor sa najvećim stepenom u podgrafu neobojenih čvorova, a zatim čvor sa najmanjim identifikatorom.

Nakon izbora čvora određuje se najmanja dozvoljena boja, zatim se ažuriraju stepeni zasićenja susednih neobojenih čvorova i postupak se ponavlja do završetka algoritma.

Prednost ove implementacije jeste jednostavnost i preglednost, dok je njen osnovni nedostatak potreba za pretragom svih neobojenih čvorova u svakom koraku izvršavanja.

4.4 Implementacija sa binarnim hipom

Radi smanjenja vremena potrebnog za izbor narednog čvora, razvijena je implementacija zasnovana na binarnom hipu. Prioritet svakog čvora određuje se na osnovu stepena zasićenja, stepena u podgrafu neobojenih čvorova i identifikatora čvora.

Pošto standardna implementacija binarnog hipa ne podržava direktnu izmenu prioriteta elemenata, primenjena je tehnika lenje invalidacije. Prilikom promene prioriteta u hip se umeće novi zapis sa ažuriranim prioritetom, dok se prethodni zapis ne uklanja odmah, već ostaje u strukturi podataka. Prilikom izdvajanja elementa sa vrha hipa, koje se izvršava u vremenu $\mathcal{O}(\log n)$, proverava se da li izdvojeni zapis predstavlja trenutno važeći prioritet odgovarajućeg čvora. Ukoliko je zapis zastareo, odbacuje se i postupak se ponavlja sve dok se ne izdvoji važeći element.

Prednost ovog pristupa je što se izbegava potreba za složenijim operacijama ažuriranja prioriteta, čime se zadržava efikasnost implementacije. Međutim, njegov nedostatak ogleda se u postepenom gomilanju zastarelih zapisa u hipu. Zbog toga broj elemenata u strukturi podataka može biti znatno veći od broja čvorova grafa, što povećava zauzeće memorije i može dovesti do većeg broja uzastopnih izdvajanja zastarelih elemenata pre pronalaženja važećeg prioriteta.

4.5 Implementacija sa Fibonačijevim hipom

Treća implementacija koristi Fibonačijev hip, koji omogućava efikasnije ažuriranje prioriteta u odnosu na binarni hip. Za razliku od implementacije zasnovane na binarnom hipu, gde se pri promeni prioriteta umeće novi zapis, ovde se za svaki čvor održava direktna referenca na odgovarajući element u hipu. U tu svrhu koristi se pomoćna mapa koja preslikava identifikator čvora na njegov element u Fibonačijevom hipu.

Prioritet čvora određen je uređenim parom $(-s, v)$, gde je s stepen zasićenja, a v identifikator čvora. Pošto se koristi minimalni hip, negiranjem stepena zasićenja postiže se da čvor sa najvećim stepenom zasićenja bude izdvojen kao element sa najmanjim ključem. Identifikator čvora služi za razrešavanje slučajeva u kojima više čvorova ima isti stepen zasićenja.

Prilikom povećanja stepena zasićenja nekog čvora nije potrebno umetati novi element u hip. Zahvaljujući direktnoj referenci na odgovarajući element, moguće je pozvati operaciju *decrease_key*, koja ažurira njegov prioritet za amortizovano konstantno vreme. Time se izbegava primena tehnike lenje invalidacije i uklanja potreba za održavanjem više zapisa koji predstavljaju isti čvor.

Nakon izbora čvora za bojenje poziva se operacija izdvajanja minimalnog elementa iz Fibonačijevog hipa, čija amortizovana vremenska složenost iznosi $\mathcal{O}(\log n)$. Nakon izdvajanja, referenca na odgovarajući element uklanja se iz pomoćne mape, čime se obezbeđuje da se u hipu u svakom trenutku nalazi tačno jedan element za svaki neobojeni čvor.

Ovakav pristup eliminiše glavni nedostatak implementacije zasnovane na binarnom hipu, odnosno gomilanje zastarelih zapisa. Broj elemenata u hipu ostaje jednak broju trenutno neobojenih čvorova, čime se smanjuje zauzeće memorije i izbegava dodatno vreme potrebno za odbacivanje nevažećih elemenata. Zahvaljujući tome ostvaruje se teorijska vremenska složenost od $\mathcal{O}(m + n \log n)$.

4.6 Poređenje implementacija

Sve implementacije koriste isti postupak dodele boja i proizvode identičan sled koraka izvršavanja. Razlika između njih ogleda se isključivo u načinu održavanja prioriteta neobojenih čvorova.

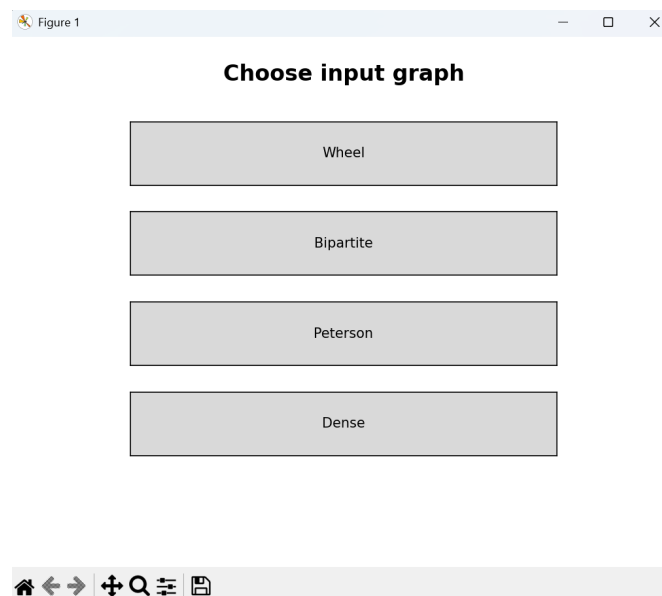
Standardna implementacija pogodna je za jednostavnije primere i lakše razumevanje rada algoritma, dok implementacije zasnovane na hipovima smanjuju vreme izbora narednog čvora i ostvaruju bolje performanse na grafovima većih dimenzija. Posebno se izdvaja implementacija sa Fibonačijevim hipom, koja postiže najpovoljniju teorijsku vremensku složenost zahvaljujući efikasnom ažuriranju prioriteta.

5 Vizuelizacija rada algoritma

Pored implementacije DSatur heuristike, razvijena je i aplikacija koja omogućava interaktivni prikaz procesa bojenja grafa. Za razliku od klasičnih implementacija koje prikazuju samo konačan rezultat, aplikacija omogućava korisniku da prati izvršavanje algoritma korak po korak i analizira odluke koje se donose tokom rada.

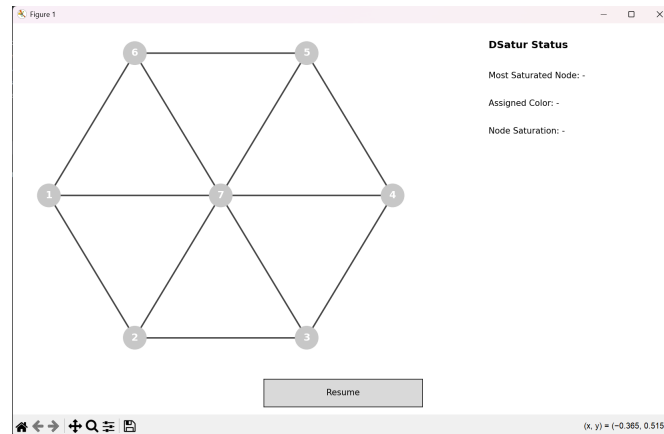
5.1 Grafički korisnički interfejs

Aplikacija poseduje grafički korisnički interfejs koji objedinjuje prikaz grafa i informacije o trenutnom stanju algoritma. Nakon pokretanja programa korisnik može odabrati jedan od unapred pripremljenih primera 1 ili zadati proizvoljnu ulaznu datoteku putem komandne linije.



Slika 1: Prozor za odabir ulaznog grafa

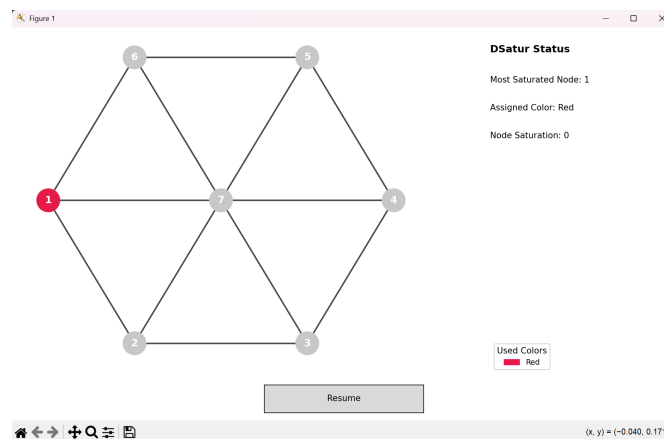
Kao što se može videti sa slike 2, glavni prozor aplikacije podeljen je na dva dela. U levom delu prikazuje se graf čiji se čvorovi boje tokom izvršavanja algoritma, dok se u desnom delu nalaze informacije o trenutno obrađenom čvoru i parametrima koji utiču na izbor narednog koraka.



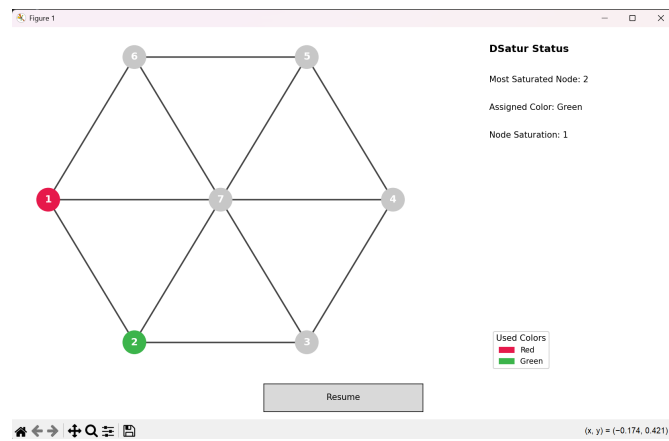
Slika 2: Primer glavnog prozora aplikacije

5.2 Animacija procesa bojenja

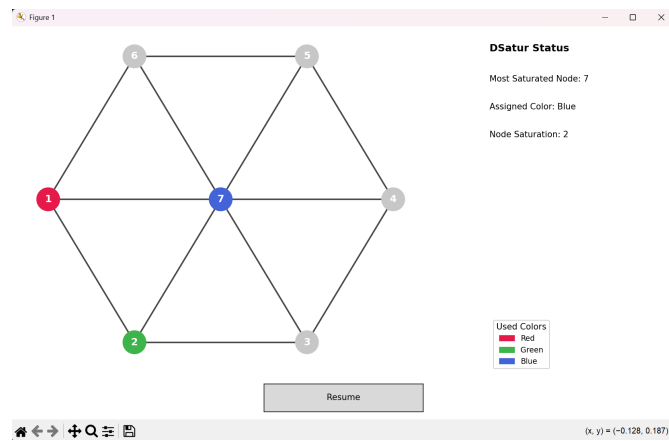
Proces bojenja prikazan je kao animacija u kojoj se svaki korak algoritma izvršava nakon unapred definisanog vremenskog intervala. Nakon izbora čvora, njegova boja se odmah ažurira u grafičkom prikazu, čime korisnik može da prati redosled kojim algoritam obrađuje čvorove. Opisani postupak je prikazan na slikama 3, 4, 5 i 6.



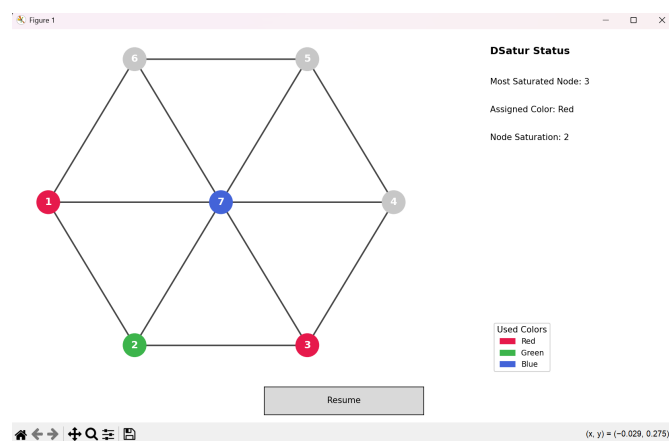
Slika 3: Primer prvog koraka bojenja grafa



Slika 4: Primer drugog koraka bojenja grafa



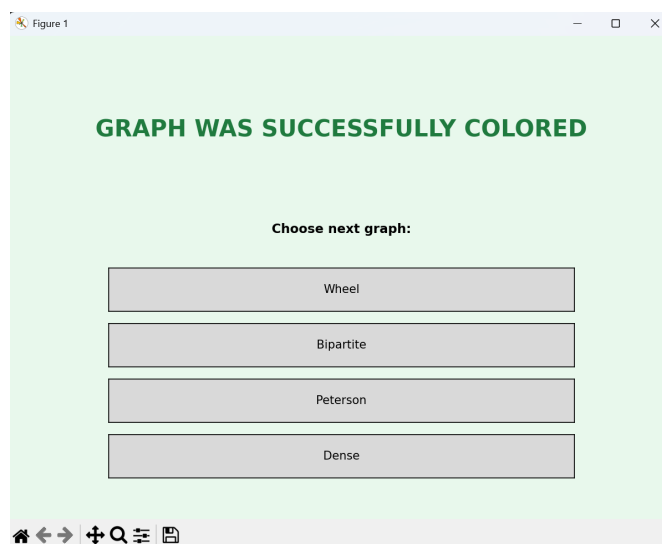
Slika 5: Primer trećeg koraka bojenja grafa



Slika 6: Primer četvrtog koraka bojenja grafa

Brzina animacije može se podesiti prilikom pokretanja programa zadavanjem odgovarajućeg parametra komandne linije. Tokom izvršavanja korisniku je omogućeno da u bilo kom trenutku pauzira ili nastavi animaciju, što olakšava detaljnu analizu pojedinačnih koraka algoritma.

Po završetku bojenja prikazuje se prozor koji omogućava izbor novog grafa bez potrebe za ponovnim pokretanjem aplikacije, što je prikazano na slici 7. Na ovaj način olakšano je poređenje rada algoritma na različitim primerima.



Slika 7: Prozor za izbor novog grafa za bojenje

5.3 Prikaz metrika algoritma

Pored vizuelnog prikaza grafa, aplikacija prikazuje i informacije koje određuju tok izvršavanja algoritma. Za svaki korak prikazuju se identifikator trenutno izabranog čvora, njegov stepen zasićenja i boja koja mu je dodeljena.

Prikaz ovih podataka omogućava korisniku da poveže promene na grafu sa odlukama koje algoritam donosi. Na taj način moguće je jasno uočiti kako se nakon svakog koraka menjaju stepeni zasićenja susednih čvorova i zbog čega je određen čvor izabran u narednoj iteraciji.

Kombinovanjem grafičkog prikaza i prikaza metrika aplikacija predstavlja koristan alat za razumevanje rada DSatur heuristike i analizu njenog ponašanja na različitim tipovima grafova.

5.4 Zanimljivosti tokom razvoja aplikacije

Tokom razvoja aplikacije identifikovano je nekoliko izazova čije je rešavanje doprinelo unapređenju implementacije i kvaliteta vizuelizacije.

Prvobitna implementacija Fibonačijevog hipa oslanjala se isključivo na operacije umećanja i izdvajanja elemenata koje pruža sama struktura podataka. Pri svakoj promeni prioriteta odgovarajući element bio je uklanjao iz hipa, nakon čega je ponovo umetnut sa ažuriranim ključem. Ovakav pristup nije koristio osnovnu prednost Fibonačijevog hipa, odnosno operaciju smanjenja ključa. U konačnoj verziji uveden je rečnik koji svakom čvoru grafa pridružuje odgovarajući element u hipu. Na taj način omogućeno je direktno pronalaženje elementa i pozivanje operacije *decrease_key*, čime je ostvarena njena amortizovana vremenska složenost od $\mathcal{O}(1)$.

Drugi izazov odnosio se na vizuelni prikaz grafa. U početnoj verziji aplikacije korišćen je raspored čvorova *spring_layout* iz biblioteke NetworkX. Iako ovaj raspored često daje estetski prihvatljive rezultate, kod pojedinih grafova dolazilo je do ukrštanja velikog broja grana, zbog čega je sticao utisak da pojedine grane povezuju čvorove koji zapravo nisu susedni. Radi postizanja preglednijeg prikaza, u konačnoj verziji primenjeni su rasporedi *shell_layout*, *kamada_kawai_layout* i *circular_layout*, u zavisnosti od karakteristika grafa koji se prikazuje. Time je znatno poboljšana čitljivost vizuelizacije i olakšano praćenje rada algoritma.

6 Zaključak

U okviru ovog rada predstavljen je problem bojenja grafova i jedna od najpoznatijih heuristika za njegovo rešavanje, DSatur algoritam. Analizirani su osnovni principi rada algoritma, kriterijumi za izbor narednog čvora i uticaj različitih struktura podataka na efikasnost njegove implementacije.

Razvijena aplikacija objedinjuje više implementacija DSatur algoritma, uključujući standardnu verziju, implementaciju zasnovanu na binarnom hipu i implementaciju koja koristi Fibonačijev hip. Upotrebom zajedničke arhitekture omogućeno je poređenje ovih implementacija uz identično ponašanje algoritma, pri čemu se razlike ogledaju isključivo u načinu održavanja prioriteta čvorova i vremenskoj složenosti izvršavanja.

Posebna vrednost razvijenog rešenja ogleda se u mogućnosti vizuelnog praćenja procesa bojenja. Prikaz izvršavanja algoritma korak po korak, zajedno sa relevantnim parametrima koji utiču na donošenje odluka, omogućava bolje razumevanje rada heuristike i predstavlja koristan alat u obrazovne svrhe.

Iako DSatur ne garantuje pronalaženje optimalnog bojenja za svaki graf, u praksi daje veoma kvalitetna rešenja uz prihvatljivo vreme izvršavanja. Zbog toga predstavlja jednu od najznačajnijih heuristika za rešavanje problema bojenja grafova i dobar primer primene odgovarajućih struktura podataka u unapređenju efikasnosti algoritama.

Literatura

- [1] Daniel Brélaz. New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256, 1979.
- [2] Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5 edition, 2017.

- [3] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.
- [4] Rhyd Lewis. *A Guide to Graph Colouring: Algorithms and Applications*. Springer, 2 edition, 2021.
- [5] Douglas B. West. *Introduction to Graph Theory*. Prentice Hall, 2 edition, 2001.